

Contexto

Até agora o Sonora guardou tudo em vetores de tamanho fixo: a Playlist tem espaço para 100 músicas, a Plataforma para 500 músicas e 500 usuários. Além disso, as relações entre as classes existiam de forma implícita, escondidas dentro desses vetores e de atributos soltos. Nesta fase você vai fazer duas coisas que andam juntas: primeiro desenhar, em UML, como as classes do Sonora se relacionam de verdade, com todos os adornos (papel, nome, multiplicidade e navegabilidade); depois trocar os vetores por ArrayList, que é a estrutura que o Java oferece para guardar uma quantidade que cresce sozinha.

O pulo do gato é perceber que as duas coisas conversam. Quando você modela um relacionamento como um para muitos, aquele lado muitos vira, no código, uma coleção. É aí que o ArrayList entra.

Parte 1 - Modelar os relacionamentos do Sonora

Antes de mexer no código, você vai produzir o diagrama de classes do Sonora com as associações devidamente detalhadas. Uma associação é o relacionamento que conecta duas classes e permite que os objetos de uma naveguem até os da outra. Cada associação deve trazer os quatro adornos vistos em aula.

Os quatro adornos (revisão rápida)

- **Papel.** O nome do lado da associação, escrito junto da classe, dizendo qual papel aquela classe cumpre no relacionamento (por exemplo, - dono, - faixas).
- **Nome.** Um verbo que descreve a natureza da ligação, com uma seta indicando a direção de leitura (por exemplo, Usuario cria Playlist).
- **Multiplicidade.** Quantos objetos de um lado se ligam a um objeto do outro lado.
- **Navegabilidade.** De qual lado para qual lado é possível caminhar. Por padrão é bidirecional; uma seta na ponta indica que a navegação vale só naquela direção.

Tabela de multiplicidades

Multiplicidade	Significado
0..1	Zero ou um objeto (relacionamento opcional, no máximo um do outro lado)
1 (ou 1..1)	Exatamente um objeto
0..*	Zero ou muitos (pode não haver nenhum)
1..*	Pelo menos um, podendo haver muitos
m..n	Entre m e n objetos (por exemplo, 3..5)

O que você vai entregar na modelagem

Desenhe o diagrama de classes do Sonora contendo, no mínimo, os quatro relacionamentos abaixo. Para cada um, defina os papéis, o nome com a direção de leitura, a multiplicidade nas duas pontas (justificando com a técnica acima) e a navegabilidade (justificando por que é bi ou unidirecional). O relacionamento entre Playlist e Musica já está resolvido no exemplo; os demais são com você.

1. Plataforma e Musica (o acervo de músicas cadastradas).

2. Plataforma e Usuario (os usuários registrados).
3. Usuario e Playlist (o usuário é dono das playlists que cria).
4. Usuario e Usuario, uma associação reflexiva: um usuário pode seguir outros usuários. Pense nos papéis - seguindo e - seguidores.

Sobre a associação reflexiva: ela conecta objetos de uma mesma classe. No Sonora, um Usuario se liga a outros objetos Usuario pela relação de seguir. Trate os dois papéis com atenção, porque a mesma classe aparece nas duas pontas.

Parte 2 - Trocar os arrays por ArrayList

Agora o código. Todo lugar onde hoje existe um array de tamanho fixo para guardar objetos vira um ArrayList. Uma instância de ArrayList guarda vários objetos, não tem tamanho limitado e recupera os elementos pela posição. Ela não guarda tipos primitivos, apenas objetos.

O que muda em cada classe

- **Playlist.** O array de músicas vira ArrayList<Musica>. Os métodos adicionar, getNaPosicao, removerNaPosicao, getDuracaoTotalSegundos e reproduzirTudo passam a usar add, get, remove e size. A contagem manual de quantidade sai de cena: quem responde isso agora é o size().
- **Plataforma.** Os arrays de músicas e de usuários viram ArrayList<Musica> e ArrayList<Usuario>. As buscas percorrem a lista com for ou for-each.
- **Usuario.** Ganha a coleção da associação reflexiva: um ArrayList<Usuario> com os usuários que este usuário segue.

Parte 3 - Implementar a associação reflexiva

Dê vida à relação de seguir usuários. Na classe Usuario, implemente:

- **seguir(Usuario outro)** - adiciona outro à lista de seguindo. Um usuário não deve seguir a si mesmo nem seguir duas vezes o mesmo usuário.
- **deixarDeSeguir(Usuario outro)** - remove outro da lista de seguindo.
- **getQuantidadeSeguindo()** - retorna quantos usuários este usuário segue.

No App, adicione ao menu as opções para um usuário seguir e deixar de seguir outro, e para listar quem um usuário segue. Trate as entradas inválidas com o que você já aprendeu sobre exceções.

Entregáveis

1. Projeto sonora-fase05, clonado da fase anterior.
2. Diagrama de classes do Sonora com os quatro relacionamentos, cada um com papel, nome com direção, multiplicidade nas duas pontas e navegabilidade. Entregue como imagem ou PDF dentro do projeto (por exemplo, docs/diagrama-classes.png).
3. Código refatorado: arrays trocados por ArrayList em Playlist e Plataforma, e a coleção da associação reflexiva em Usuario.
4. A funcionalidade de seguir e deixar de seguir usuários funcionando pelo menu do App.